
Agents and Environments

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

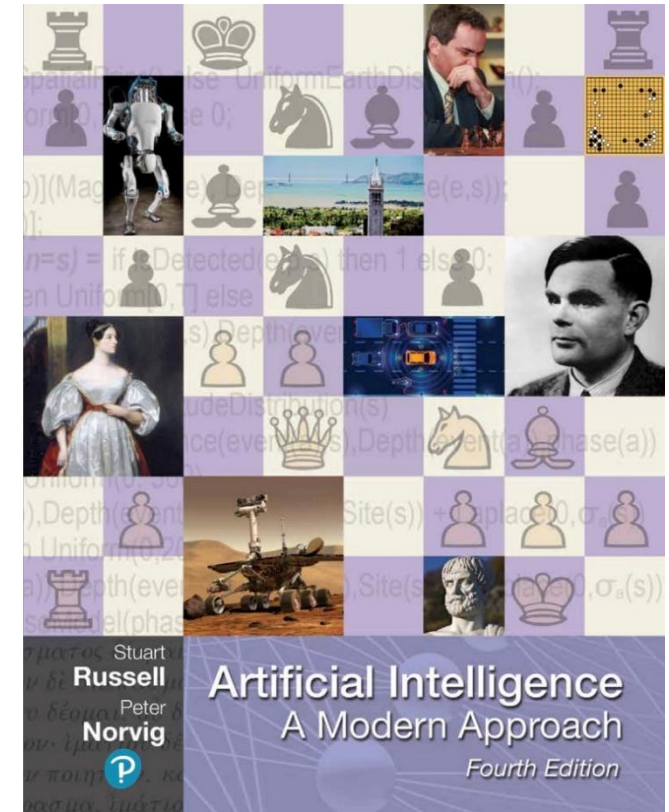
Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Some figures adapted from CS188: Introduction to Artificial Intelligence, UC Berkeley (Dan Klein, Pieter Abbeel) — ai.berkeley.edu
- Some figures are obtained from the [AIMA Book](#)
- Some content adapted from Dr. ElSayed El-Alfy

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Resources & Reading

- Textbook website: <http://aima.eecs.berkeley.edu/>
- Read Chapter 2 in AIMI 4/E

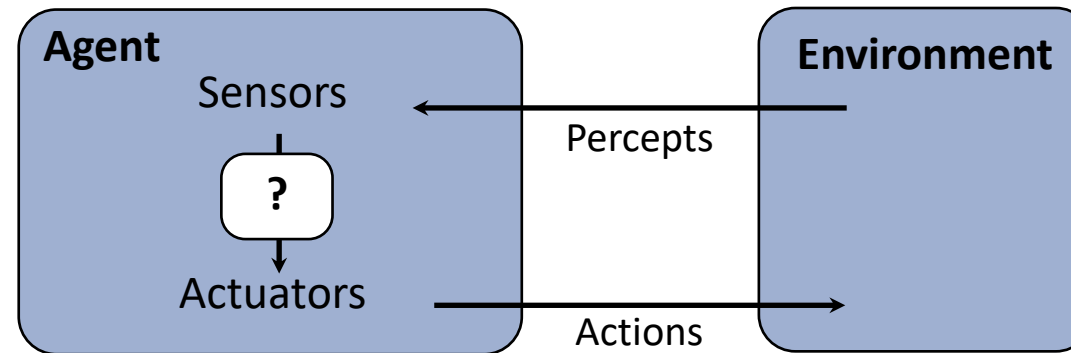


Outline

- Agents and Rationality – What is good behavior?
- Types of Environments – PEAS description
- Programming an environment
- Summary

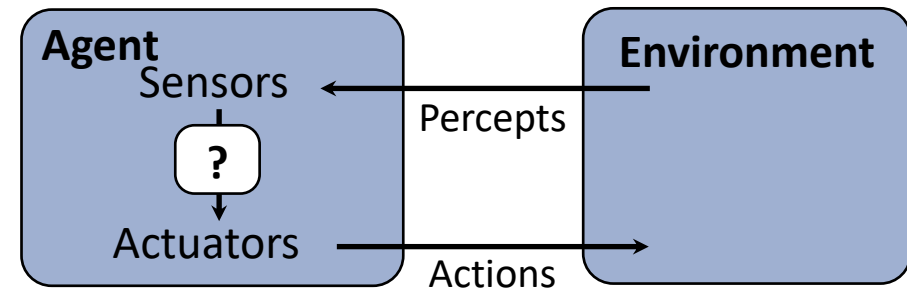
Agents and Rationality – What is good behavior?

What is an Agent?



- “An agent is anything that can be viewed as **perceiving** its **environment** through **sensors** and **acting** upon that environment through **actuators**” – [R&N book]
- Humans are agents:
 - Sensors = eyes, ears, skin, nose, tongue
 - Actuators = muscles, secretions, hands, legs, etc.
- Robotic agent:
 - Sensors: cameras, infrared, ultrasound, etc.
 - Actuators: Various motors, robotic arm, wheels, etc.
- Software agent:
 - Sensors: user mouse/keyboard input, receiving data/files
 - Actuators: sending data, writing data, updating content, etc.

Agent percepts and function



- An agent's **percept** is the content of what it perceives with its sensors.
- **Percept sequence P^*** : complete history of all percepts that the agent has ever perceived.
- **Agent function**: maps a given percept sequence P^* to an action A .

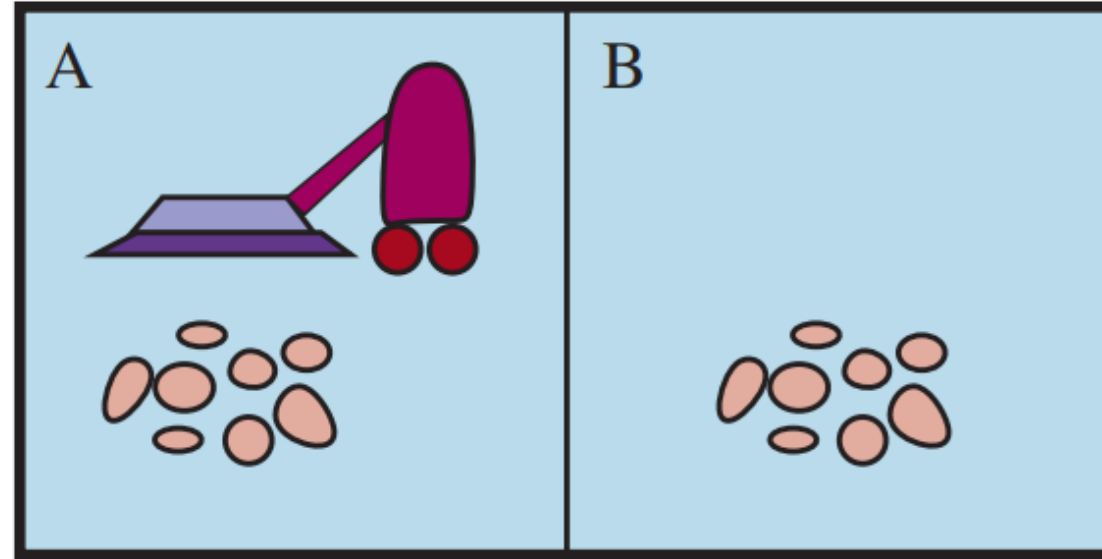
$$f: P^* \rightarrow A$$

- The agent function is **practically implemented** as an **agent program** that runs on some computing device with physical sensors and actuators (**architecture**).

Agent = architecture + program

- For most of this class we focus on the design of agent programs. i.e. the **implementation** of mapping percepts to actions.

Example: Vacuum Cleaner Agent [1]



- Two locations A and B. Locations can be dirty or clean. Agent starts in location A. The agent can:
 - percept content: [location, status]. For example [A, dirty], [B, clean]
 - Action: Left, Right, Suck, NOOP
- How many possible *configurations* are there for a single **percept**?
- Can you think of possible **percept sequences** for this environment?
- Can you think of an **agent function** for this agent? [does it achieve the goal?]
- Can you write pseudocode (**agent program**) for the agent function?

Example: Vacuum Cleaner Agent [2]

*Example of an agent program that
returns what action to take*

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action  
  if status = Dirty then return Suck  
  else if location = A then return Right  
  else if location = B then return Left
```

- Many ways to select actions in the agent function, but which one is “correct” or “good”? That is, what is a “**right**” agent function.
- Recall the concept of **rational** agents. The right action is the rational action.

“Good” behavior: Rationality [1]

- We use the following rational agent definition:
A **rational agent** chooses the action that **maximizes** the **expected value** of the performance measure given the **percept sequence to date** (*and whatever **built-in knowledge** the agent has*).
- Agent rationality (doing the ‘right’ action) depends on:
 - **performance measure**: that defines criterion of success
 - **environment knowledge**: what it knows about its environment
 - **percept sequence**: everything it has perceived so far
 - **actions**: that it can perform (behavior)
- Built-in knowledge are known as **priors** or **assumptions** about the environment the agent operates in. This allows us to build transition-models computationally (as well as other properties).

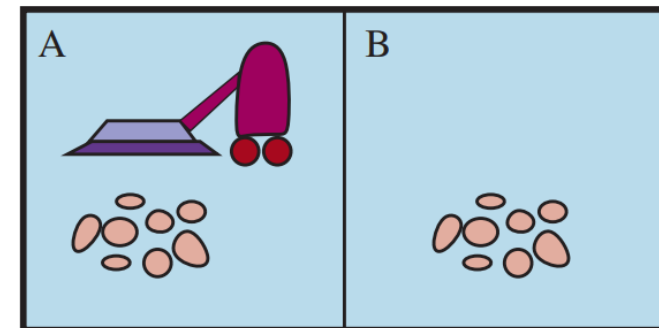
“Good” behavior: Rationality [2]

- Rationality $\neq$ perfect success. Rationality is maximizing *expected* performance (uncertainty); perfection is maximizing actual performance.
- Related to uncertainty in environments:
 - Rational agents may perform actions to obtain useful information (*information gathering* or *exploration*) which is particularly helpful in uncertain/unknown environments.
 - This helps the agent *learn* about the environment mechanics. With more *experience*, it can maximize its expected performance.
 - We will study how to model uncertainty using statistics/probability.
 - We will see examples of this in game-playing and Bayesian networks.
- Rational agents may interact with other agents: multi-agent systems, cooperative/collaborative vs. competitive, conflict resolution.

Types of Environments – PEAS description

The task environment - PEAS

- To design a rational agent, we must first specify the **task environment (PEAS)**:
 - Performance
 - Environment
 - Actuators
 - Sensors
- The task environment can be thought of as the problem, and the rational agent is the solution.
- For the vacuum cleaner agent:
 - Performance: 1 point per clean location per time step, for $t = 1, \dots, T$
 - Environment: locations A and B, clean and dirty
 - Actuators: left, right, suck, NoOp
 - Sensors: sensor to detect clean status



Example: Autonomous Taxi

- Performance:

Safe, fast, comfortable trip, maximize profits, etc.

- Environment:

Roads, other traffic, pedestrians, customers, etc.

- Actuators:

Steering wheel, accelerator, brake, signal, horn, etc.

- Sensors:

Cameras, sonar, speedometer, GPS, odometer, engine sensors, etc.



More examples from textbook

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments	Touchscreen/voice entry of symptoms and findings
Satellite image analysis system	Correct categorization of objects, terrain	Orbiting satellite, downlink, weather	Display of scene categorization	High-resolution digital camera
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, tactile and joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, raw materials, operators	Valves, pumps, heaters, stirrers, displays	Temperature, pressure, flow, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, feedback, speech	Keyboard entry, voice

Note these specifications are given by the book. Depending on the actual real-world problem you are facing, you may give another PEAS specification that is more accurate.

Dimensions/Properties of the task environment [1]

- Various task environments can be categorized along a set of dimensions/properties. ***It is a design consideration which influences agent and program design.***
- (Book perspective) There are about 7 dimensions for the task environment:
 1. Fully observable vs. partially observable
 - Observable: Sensors give complete state of the environment at each point in time. Example is chess.
 - Partially observable example: vacuum cleaner agent cannot sense the status of adjacent location.
 2. Single agent vs. multi agent
 - An agent is operating by itself in an environment.
 - Multi agents can be competitive (one wants to beat the other) or cooperative (maximizes the scores for everybody) some environments can be cooperative and competitive at the same time.
 - Can we model a multi-agent environment as single-agent?
 3. Deterministic vs. non-deterministic (stochastic)
 - Deterministic: next environment state is completely determined by current state and action (no uncertainty).
 - Non-deterministic (stochastic): next environment state is defined by a probability depending on the action performed and the current state. I.e. Performing action X in state A, will transition to state B with 30% chance or state C with 70% chance.
 - Most real-world complex examples are treated as non-deterministic.

Dimensions/Properties of the task environment [2]

4. Episodic vs sequential (the agent-environment interaction occurs throughout an episode)
 - Episodic: next episode does **not depend** on actions taken in previous episodes (independence). Example: prediction tasks in machine learning, one game of chess and another game of chess, etc.
 - Sequential: current action could affect all future actions. Example: the actions taken *within* a single chess match are sequential.
5. Dynamic vs static
 - Static: the environment does **not** change while the agent is thinking (computing) an action.
 - Dynamic: the environment **can** change while the agent is thinking of (computing) what action to do.
 - Semi-dynamic: environment does not change with the passage of time, but the agent's performance score does. Like Chess with a timer.
6. Discrete vs continuous
 - Discrete: **finite** number of environment states and **finite** agent actions.
 - Continuous: infinite number of environment states and/or infinite number of actions.
7. Known vs unknown (is this different from deterministic vs stochastic?)
 - Known: the *dynamics/physics/model* of the environment is known. This means that the outcome of actions are specified completely. Even with uncertainty, we can use probability model to get expected outcome.
 - Unknown: the agent will have to explore the outcomes of actions and build a *transition model* over time.

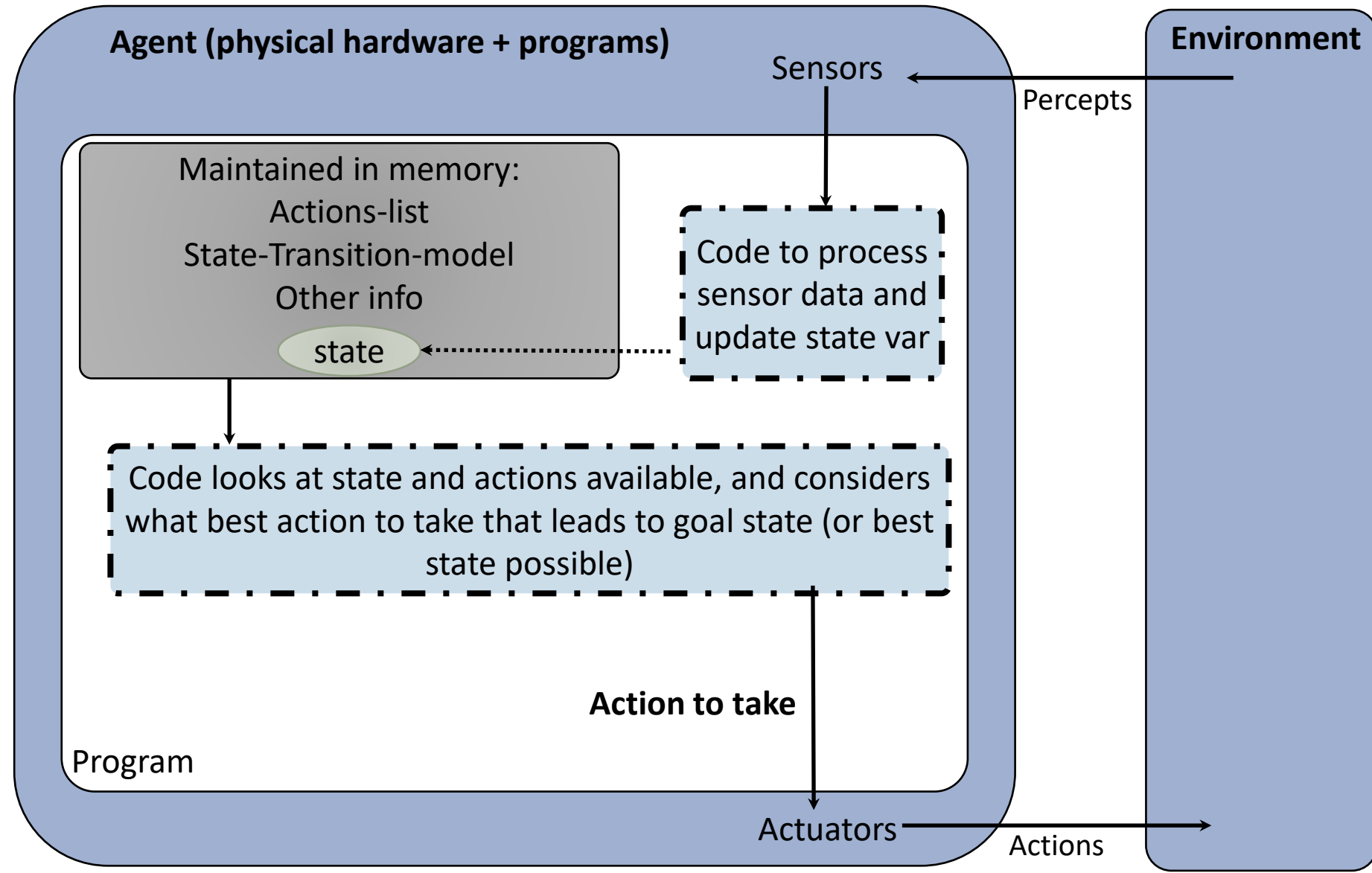
Example of task environment categorization from textbook

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

- Hardest case is partially observable, multi-agent, nondeterministic, sequential, dynamic, continuous, unknown. Like automated taxi driver.

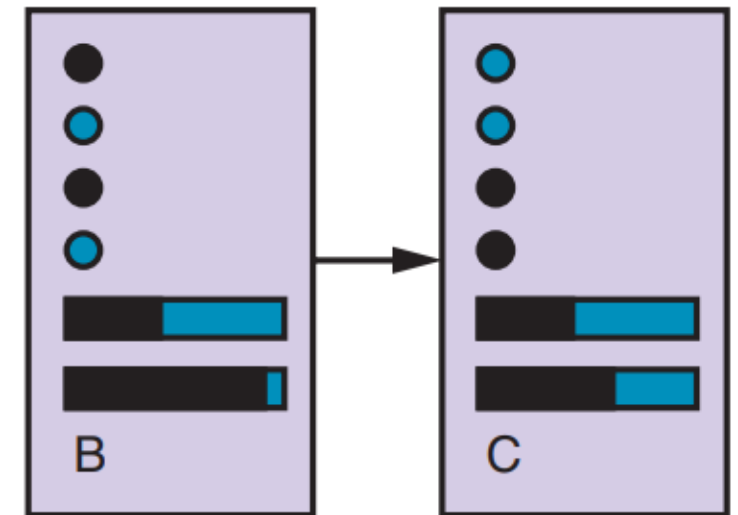
State Representation (Programmatically)

- This is an *example* of what an agent can do programmatically.
- One important info that the agent can maintain in memory is a *computational representation of the environment (state)*.
- The agent can then consider *computationally* what action to do.
- If you know how the environment operates, you can code the state-transition model to help with action selection.
- Focus in this course is the program.



How to represent state programmatically?

- You have a lot freedom in how to represent states programmatically.
- Think of state as the current situation of the environment + agent dynamic properties.
- The easiest to understand representation is:
 - Represent state as an object (classes from OOP) or list with variables.
 - Lists are enough for majority of our use cases in this course.
 - Each variable represents a property of the environment or agent (or other info that *changes*).
 - Technically you can also include properties that do not change (static), however, this just consume extra space. Instead, you can *hard-code* this information in the code or specify them as part of the agent program's static variables (not part of the state list).
 - We will see various examples in the coming lectures.
- Simple example in the figure on the right for some problem:
 - A state is a list of 6 variables (4 Boolean and 2 float).
 - A transition from state to state is a change in variable values.
 - A state is unique (state *B* here is not the same as state *C*)
- On some problems, you can use less or more complex representations.



State programmatically Example [1]

- Two locations A and B. Locations can be dirty or clean. Agent starts in location A.
- Let's say that the agent can sense the status of **both** locations A and B at any time.
- Actions: Left, Right, Suck, NOOP
- Programmatically, the changing properties are current agent-location and statuses of A and B. So, one way we can represent the state is as a list:

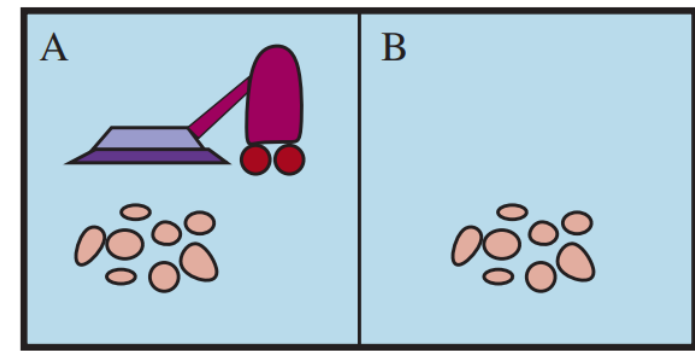
$$state = [loc, \quad statusA, \quad statusB]$$

$loc \in \{A, B\}$ represents current agent location.

$statusA \in \{dirty, clean\}$ represents A status.

$statusB \in \{dirty, clean\}$ represents B status.

- What about the transition logic? We can hard-code it.



State programmatically Example [2]

- What about the transition logic?
Since this knowledge is available for this problem, we can hard-code it.
- We'll see more examples on various problems in the coming lectures.

```
# computationally model the environment transitions
def state_transition(state, action):
    loc = state[0]
    statusA = state[1]
    statusB = state[2]

    next_state = []

    if action == 'Left':
        if loc == 'A': # edge-case: agent stays in place
            next_state = ['A', statusA, statusB]
        elif loc == 'B':
            next_state = ['A', statusA, statusB]

    if action == 'Right':
        if loc == 'A':
            next_state = ['B', statusA, statusB]
        elif loc == 'B': # edge-case: agent stays in place
            next_state = ['B', statusA, statusB]

    if action == 'Suck':
        if loc == 'A':
            next_state = [loc, 'clean', statusB]
        elif loc == 'B': # edge-case: agent stays in place
            next_state = [loc, statusA, 'clean']

    if action == 'NOOP': # no change
        next_state = [loc, statusA, statusB]

    return next_state
```

Practice

- Problem info:
 - You have an environment with $N \times M$ grids (it is a 2D grid). N and M are given to you (integers).
 - The agent can move up, left, right, down. The agent can paint the grid under him red or blue color.
 - Goal is to paint half the grids blue and half the grids red. Assume N and M are even integers.
 - At the start, the agent starts in top-leftmost grid position. All grids are uncolored.
 - The agent can sense the status of all grids at all timesteps.
- Questions:
 - Give an *efficient* representation for the state of this environment. By efficient, use as few variables as possible. Specify the domain of each variable.
 - Using your representation, what is the initial state of this problem?
 - What about N and M ? Do they **need** to be part of the state?
 - Write the pseudocode for the state-transition-model.
 - What is the goal condition for your state representation? Use logical notation.

Summary

Summary

- An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.
- Practically, the agent is a physical entity that has an agent program, sensors, and actuators.
- The agent program runs on some computing .
- Rational agents choose actions that maximize their *expected* utility.
- Agent design is influenced by the PEAS description (task environment).
- Task environment can be categorized along 7 dimensions.
- Representing the state of a problem programmatically is an important part of building the agent program. Typically, a state is modelled as a list of changing variables/properties.